#10 구역 등록 — 코드 상세 설명서

출처: Notion `[#10] 코드리뷰 · 구역 등록` (PR #107). 아래 코드는 Notion에 적힌 코드다.

1. 전체 흐름

```
관리자: 이름 + 그린 다각형
   → ZoneController              경비원: 요청 받기, 201 응답
   → ZoneRegisterService         센터장: 순서대로 시키기, 전부 되거나 전부 취소
        ├→ ZonePreviewService.calculate (#9)   다각형 검사, 소속 동, 넓이, 중심점
        ├→ ZoneRepository.existsBy...          같은 동에 같은 이름 있나?
        │       있음 → 409 DUPLICATE_ZONE_NAME
        └→ ZoneRepository.save                 zone 저장
   → 201 {id, dongId, name, areaM2}
```

2. 명세가 요구한 것 vs 구현

명세 (FNC-A-014)	구현
1. FNC-A-013 계산 다시 수행(서버 값 사용)	`ZonePreviewService.calculate()` 재사용
2. 같은 동의 삭제되지 않은 구역과 이름 중복 확인	`existsByDongIdAndNameAndDeletedAtIsNull`
3. 다각형 내부 점(getInteriorPoint)을 center_lat/lng로	계산 단계에서 구한 점을 소수 6자리로 저장
4. zone 저장	`ZoneRepository.save`
5. 작업 기록(ZONE_CHANGE)	**미구현** (#6 대기)
예외: 이름 중복 409 DUPLICATE_ZONE_NAME	`DuplicateZoneNameException` + `ErrorCode.DUPLICATE_ZONE_NAME`
API: 201 {id, dongId, name, areaM2}	`ZoneCreateResponse`

3. 코드

3-1. DB 표: V4__create_zone.sql

```
CREATE TABLE zone (
    id BIGINT NOT NULL AUTO_INCREMENT COMMENT '구역 번호',
    dong_id BIGINT NOT NULL COMMENT '소속 동(region.id). 다각형 중심점이 속한 읍·면·동으로 자동 판정',
    name VARCHAR(50) NOT NULL COMMENT '구역 이름. 같은 동의 삭제되지 않은 구역끼리 중복 불가(앱에서 확인)',
    area_m2 DECIMAL(10, 2) NOT NULL COMMENT '면적(㎡). 다각형에서 자동 계산',
    boundary JSON NOT NULL COMMENT '구역 다각형(GeoJSON, WGS84)',
    center_lat DECIMAL(9, 6) NOT NULL COMMENT '구역 중심점 위도(다각형 안의 점)',
    center_lng DECIMAL(9, 6) NOT NULL COMMENT '구역 중심점 경도',
    deleted_at DATETIME NULL COMMENT '삭제 시각(NULL이면 사용 중)',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '등록 시각',
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '마지막 수정 시각',
    CONSTRAINT zone_pk PRIMARY KEY (id),
    CONSTRAINT zone_fk_dong FOREIGN KEY (dong_id) REFERENCES region (id)
) ENGINE = InnoDB;

CREATE INDEX zone_ix_dong_deleted ON zone (dong_id, deleted_at);
```

부분	쉬운 뜻
`FOREIGN KEY (dong_id) REFERENCES region (id)`	구역은 반드시 실제 있는 동에 속해야 한다는 약속. 없는 동 번호를 넣으면 DB가 거부한다

부분	쉬운 뜻
`area_m2 DECIMAL(10, 2)`	숫자 10자리 중 소수 2자리. 약 1억 ㎡(100㎢) 미만만 저장된다 (리뷰 R4)
`deleted_at`	삭제하면 지우지 않고 시각만 적는 칸(소프트 삭제). NULL이면 쓰는 중
`updated_at ... ON UPDATE CURRENT_TIMESTAMP`	행이 바뀔 때마다 DB가 알아서 시각을 갱신한다
`zone_ix_dong_deleted (dong_id, deleted_at)`	"이 동의 삭제 안 된 구역"을 빨리 찾는 색인 (이름 중복 확인이 쓴다)
이름에 UNIQUE가 없다	명세가 "삭제된 구역 이름을 다시 쓸 수 있게 DB 유일 조건은 걸지 않음"이라고 했다. 중복 확인은 앱이 한다 (리뷰 R3)

3-2. 표의 Java 버전: Zone.java (common)

```
@Entity
@Table(name = "zone")
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class Zone {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // region 도메인의 엔티티를 직접 들지 않고 번호만 둔다(다른 도메인 엔티티 직접 사용 금지)
    @Column(name = "dong_id", nullable = false)
    private Long dongId;

    @Column(nullable = false, length = 50)
    private String name;

    @Column(name = "area_m2", nullable = false, precision = 10, scale = 2)
    private BigDecimal areaM2;

    @JdbcTypeCode(SqlTypes.JSON)
    @Column(nullable = false)
    private String boundary;

    @Column(name = "center_lat", nullable = false, precision = 9, scale = 6)
    private BigDecimal centerLat;

    @Column(name = "center_lng", nullable = false, precision = 9, scale = 6)
    private BigDecimal centerLng;

    @Column(name = "deleted_at")
    private LocalDateTime deletedAt;

    // 아래 두 시각은 DB 기본값(CURRENT_TIMESTAMP, updated_at은 ON UPDATE)이 채운다
    @Generated(event = EventType.INSERT)
    @Column(name = "created_at", insertable = false, updatable = false)
    private LocalDateTime createdAt;

    @Generated(event = {EventType.INSERT, EventType.UPDATE})
    @Column(name = "updated_at", insertable = false, updatable = false)
    private LocalDateTime updatedAt;

    public Zone(Long dongId, String name, BigDecimal areaM2, String boundary,
            BigDecimal centerLat, BigDecimal centerLng) { ... }
}
```

부분	쉬운 뜻
`private Long dongId`	동 정보를 통째로 들지 않고 **동 번호만** 가진다. 팀 규칙이 "다른 영역(region)의 엔티티를 직접 쓰지 말라"고 해서다

부분	쉬운 뜻
`@Generated(event = INSERT)` (created_at)	저장할 때 DB가 채운 값을 자바 쪽에서 다시 읽어 온다. 자바가 직접 넣지 않는다 (`insertable = false`)
`@Generated(event = {INSERT, UPDATE})` (updated_at)	수정할 때도 DB가 시각을 갱신하므로 UPDATE도 포함. 이후 #15(구역 수정)에서 쓰인다
`@NoArgsConstructor(PROTECTED)`, setter 없음	JPA용 빈 생성자만 숨겨 두고, 값을 바꾸는 길은 만들 때뿐이다

#7의 `Region` 과 다른 점: `Region` 은 부모를 객체(`@ManyToOne`)로 들지만, `Zone` 은 동을 **번호만** 들고 있다. 같은 도메인 안(region ↔ region)은 객체로 연결해도 되지만, 다른 도메인(zone → region)은 번호만 쓴다.

3-3. ZoneRepository (common)

```
public interface ZoneRepository extends JpaRepository<Zone, Long> {

    /** 같은 동의 삭제되지 않은 구역 중 이름이 같은 것이 있는지 본다(FNC-A-014 처리 순서 2). */
    boolean existsByDongIdAndNameAndDeletedAtIsNull(Long dongId, String name);
}
```

메서드 이름이 곧 질문이다: **exists**(있나?) **By DongId**(동 번호가 같고) **And Name**(이름이 같고) **And DeletedAtIsNull**(삭제 시각이 비어 있는) 구역. 스프링이 이름을 읽어 SQL을 만든다.

3-4. 요청·응답·오류 상자

```
// ZoneCreateRequest — boundary는 GeoJSON Polygon. 모양 검증은 service가 한다(INVALID_POLYGON).
public record ZoneCreateRequest(
        @NotBlank(message = "필수입니다.") @Size(max = 50, message = "50자 이하로 입력해 주세요.") String name,
        @NotNull(message = "필수입니다.") JsonNode boundary) {}

// ZoneCreateResponse
public record ZoneCreateResponse(Long id, Long dongId, String name, double areaM2) {}

// DuplicateZoneNameException
public class DuplicateZoneNameException extends BusinessException {
    public DuplicateZoneNameException(String name) {
        super(ErrorCode.DUPLICATE_ZONE_NAME, "같은 동에 '" + name + "' 구역이 이미 있습니다.");
    }
}

// ErrorCode 에 한 줄 추가
DUPLICATE_ZONE_NAME(HttpStatus.CONFLICT, "같은 동에 같은 이름의 구역이 있습니다."),
```

- 요청 상자는 `name` 과 `boundary` 만 받는다. 화면이 `areaM2` 나 `dongId` 를 같이 보내도 **받는 칸이 없어 무시**된다.
- `@NotBlank` : 비었거나 공백뿐이면 거부. `@Size(max = 50)` : DB 칸(50자)을 넘기지 않게 막는다. 거부되면 공통 오류 처리가 `400 INVALID_INPUT` 과 어느 칸이 문제인지(`fieldErrors`)를 돌려준다.
- 중복 오류는 `BusinessException` 을 상속해서, 던지기만 하면 공통 오류 처리가 `409 {code: DUPLICATE_ZONE_NAME, message}` 로 바꿔 준다. #9에서 쓰던 임시 핸들러가 필요 없다.

3-5. 센터장: ZoneRegisterService (핵심)

```
@Service
@RequiredArgsConstructor
public class ZoneRegisterService {

    private static final int AREA_SCALE = 2;

    private final ZonePreviewService zonePreviewService;
    private final ZoneRepository zoneRepository;

    @Transactional
    public ZoneCreateResponse register(String name, JsonNode boundary) {
```

```
        String trimmedName = name.trim();
        // 처리 순서 1: 화면이 보낸 값이 아니라 서버가 다시 계산한 값을 쓴다(검증 오류도 여기서 난다)
        ZoneCalculation calculation = zonePreviewService.calculate(boundary);
        // 처리 순서 2: 같은 동의 삭제되지 않은 구역과 이름 중복 확인
        if (zoneRepository.existsByDongIdAndNameAndDeletedAtIsNull(calculation.dongId(), trimmedName)) {
            throw new DuplicateZoneNameException(trimmedName);
        }
        // 처리 순서 3·4: 중심점은 계산 때 구한 다각형 안의 점. zone 저장
        Zone zone = zoneRepository.save(new Zone(
                calculation.dongId(),
                trimmedName,
                BigDecimal.valueOf(calculation.areaM2()).setScale(AREA_SCALE, RoundingMode.HALF_UP),
                boundary.toString(),
                calculation.centerLat(),
                calculation.centerLng()));
        // 처리 순서 5(작업 기록 ZONE_CHANGE)는 작업 기록 저장(FNC-C-026, #6)이 들어오면 여기서 호출한다
        return new ZoneCreateResponse(zone.getId(), zone.getDongId(), zone.getName(), calculation.areaM2());
    }
}
```

줄	쉬운 뜻
`@Transactional`	메서드 전체가 한 묶음. 중간에 오류가 나면 저장한 것도 전부 취소된다. 나중에 작업 기록 저장이 붙으면 "기록 저장 실패 → 구역 등록도 취소" 규칙이 저절로 지켜진다
`name.trim()`	앞뒤 공백을 지운다. "삼청동 1구역 "과 "삼청동 1구역"이 서로 다른 이름이 되는 것을 막는다
`zonePreviewService.calculate(boundary)`	#9의 계산기를 다시 쓴다. 다각형이 잘못됐거나 서비스 지역 밖이면 여기서 400 오류가 나서 아래는 실행되지 않는다. 화면 값은 쓰지 않는다 (명세: 서버 값 사용)
`existsBy...(calculation.dongId(), trimmedName)`	"이 동에 삭제 안 된 같은 이름이 있나?" 있으면 409. **동 번호는 서버가 계산한 값**을 쓴다
`BigDecimal.valueOf(...).setScale(2, HALF_UP)`	DB 칸이 소수 2자리라 맞춘다
`boundary.toString()`	받은 다각형(JSON)을 글자 그대로 저장한다
`return new ZoneCreateResponse(...)`	저장 후 DB가 정한 번호(`zone.getId()`)와 서버가 계산한 값을 응답으로 준다

순서가 중요한 이유: 계산(검증) → 중복 확인 → 저장. 도형이 잘못됐으면 이름 중복 확인까지 가지도 않고 저장도 안 한다.

3-6. 계산기에 "한 번에 계산" 추가: ZonePreviewService (#9 파일 수정)

```
@Transactional(readOnly = true)
public ZonePreviewResponse preview(JsonNode boundary) {
    ZoneCalculation calculation = calculate(boundary);
    return new ZonePreviewResponse(calculation.areaM2(), calculation.dongId(), calculation.dongName());
}

/** 검증 → 소속 동 → 면적 → 중심점을 한 번에 계산한다. 구역 등록이 서버 값을 쓰려고 다시 호출한다. */
@Transactional(readOnly = true)
public ZoneCalculation calculate(JsonNode boundary) {
    Polygon polygon = polygonReader.read(boundary);
    Point interiorPoint = polygon.getInteriorPoint();
    Region dong = findDongContaining(interiorPoint);
    double areaM2 = BigDecimal.valueOf(areaCalculator.areaM2(polygon, interiorPoint))
            .setScale(AREA_SCALE, RoundingMode.HALF_UP)
            .doubleValue();
    return new ZoneCalculation(areaM2, dong.getId(), dong.getName(),
            scaled(interiorPoint.getY()), scaled(interiorPoint.getX()));
}
```

- 원래 `preview()`가 하던 계산을 `calculate()`로 옮기고, `preview()`는 그 결과에서 필요한 세 값만 골라 돌려준다. **미리보기 API의 응답은 그대로**다. (#9 테스트 9개가 그대로 통과)
- `calculate()`는 중심점도 같이 돌려준다. 넓이를 재던 "안쪽 점"이 곧 구역의 중심점이라 같은 점을 한 번만 구한다. (`ZoneCalculation` 상자: 넓이·동 번호·동 이름·중심 위도·중심 경도)

- `y`가 위도, `x`가 경도다. (`scaled(getY())` = 위도)

3-7. 경비원: ZoneController

```
@RestController
@RequestMapping("/api/v1/manage/zones")
@RequiredArgsConstructor
public class ZoneController {

    private final ZoneRegisterService zoneRegisterService;

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public ZoneCreateResponse register(@Valid @RequestBody ZoneCreateRequest request) {
        return zoneRegisterService.register(request.name(), request.boundary());
    }
}
```

- `@PostMapping` : `POST /api/v1/manage/zones` (관리자 앱은 앞에 `/admin`이 붙는다). #9의 미리보기와 주소 앞부분만 같고 `/preview`가 있느냐로 갈린다.
- `@ResponseStatus(CREATED)` : 성공하면 200이 아니라 **201**(새로 만들었음).
- `@Valid` : 요청 상자의 `@NotBlank`·`@Size`·`@NotNull`을 검사한다. 컨트롤러는 받아서 서비스에 넘기기만 한다.

4. 테스트 12개 (전부 통과)

테스트	확인하는 것
`register_createsZone`	201과 `{id, dongId, name, areaM2}`, DB 행(이름·소속 동 삼청동·넓이 약 39,190㎡ ±1%·삭제 시각 없음·등록/수정 시각 채워짐)
`register_centerIsInsidePolygon`	저장된 중심점이 그린 사각형 안
`register_ignoresClientCalculatedValues`	요청에 `areaM2:1, dongId:999999`를 넣어도 서버 계산값이 저장됨
`register_trimsName`	" 삼청동 1구역 " → "삼청동 1구역"
`register_duplicateNameInSameDong_conflict`	같은 동 같은 이름 → 409 `DUPLICATE_ZONE_NAME`, 저장 1건 유지
`register_sameNameInOtherDong_allowed`	삼청동과 소공동에 같은 이름 "1구역" → 둘 다 201
`register_nameOfDeletedZone_allowed`	삭제 표시된 구역의 이름을 다시 써서 201
`register_invalidPolygon_badRequest`	나비넥타이 모양 → 400 `INVALID_POLYGON`, 0건 저장
`register_outsideServiceArea_badRequest`	서비스 지역 밖 → 400 `OUT_OF_SERVICE_AREA`, 0건 저장
`register_blankName_badRequest`	공백 이름 → 400 `INVALID_INPUT`, `fieldErrors`에 `name`
`register_nameTooLong_badRequest`	51자 이름 → 400 `INVALID_INPUT`
`register_missingBoundary_badRequest`	`boundary` 없음 → 400, `fieldErrors`에 `boundary`

테스트는 H2(MySQL 모드)에 #7의 샘플 경계 파일을 적재한 뒤 MockMvc로 요청한다. 실행 전마다 `zone` → `region` 순서로 비운다. `zone`이 `region`을 가리키는 외래키라 **순서가 중요**하다. 테스트가 끝난 뒤에도 `zone` 행을 지운다(`@AfterEach`).

5. 왜 이렇게 만든 게 맞는가

결정	이유	근거
#9 계산(`calculate`)을 다시 호출	화면이 보낸 넓이·동은 조작될 수 있고 미리보기 이후 데이터가 바뀔 수도 있다. 같은 계산을 두 번 만들지 않는다	명세: "FNC-A-013 계산 다시 수행 (서버 값 사용)"

결정	이유	근거
요청 상자에 `name`·`boundary`만	클라이언트 값이 들어올 길 자체를 없앤다	API 명세 비고: "면적·동은 서버가 계산"
이름 중복 확인은 동 번호 + 이름 + 삭제 안 됨	다른 동은 같은 이름 허용, 삭제된 구역 이름 재사용 허용	tables.md zone.name 비고
`Zone`은 `dongId`(번호)만 보관	다른 도메인 엔티티 직접 사용 금지	package-structure.md
엔티티·리포지토리는 common, API는 admin `zone`	사용자 앱도 구역을 읽는다	package-structure.md
중복 오류는 `BusinessException` + `ErrorCode`	공통 오류 처리가 응답을 만들어 주므로 구역 전용 핸들러가 필요 없다	FNC-C-027
새 마이그레이션 `V4`	dev에 합쳐진 V파일은 수정하지 않는다	db-migration 규칙

6. 선배가 물을 만한 질문과 답

질문	답
"미리보기가 이미 계산했는데 등록할 때 왜 또 계산해?"	화면이 보낸 값은 조작되거나 틀릴 수 있고, 미리보기 이후 데이터가 바뀔 수도 있어서 서버가 다시 계산한 값만 쓴다. 명세도 "서버 값 사용"이라고 했다.
"계산 코드를 새로 짠 거야?"	아니다. #9의 계산을 `calculate()`로 뽑아서 재사용했고, 미리보기 응답은 그대로다.
"이름 중복은 왜 DB UNIQUE로 안 막아?"	삭제된 구역의 이름을 다시 쓸 수 있게 하려는 명세 때문이다. UNIQUE를 걸면 삭제된 이름도 못 쓴다. 대신 동시 등록 시 중복이 생길 수 있다는 약점이 있다.
"`Zone`이 `Region` 객체를 안 들고 번호만 갖는 이유는?"	다른 도메인의 엔티티를 직접 쓰지 말라는 팀 규칙이다. 동 이름이 필요하면 region 쪽 서비스에 물어본다.
"`@Transactional`은 왜 붙였나?"	중간에 오류가 나면 저장한 것도 취소하려고. 나중에 작업 기록 저장이 붙으면 "기록 실패 → 구역 등록도 취소"가 저절로 지켜진다.
"작업 기록은 왜 없어?"	#6이 다른 팀원 담당이고 아직 시작 전이라 `work_history`와 저장 함수가 dev에 없다. 코드에 연결할 자리를 남겨 뒀다.
"마이그레이션 번호는 왜 V4야?"	리뷰 요청으로 바꿨다. dev에 #108이 V3를 이미 써서 번호를 올렸다. dev에 합쳐진 V파일은 수정하지 않는 규칙이다.
"테스트가 끝나면 왜 `zone`을 지워?"	안 지우면 뒤에 도는 지역 테스트가 `region`을 지우다 외래키 오류로 실패한다. 실행 순서 덕에 우연히 통과하던 잠복 문제였다.

7. 명세 대조 (FNC-A-014)

명세 항목	구현	확인
1. 계산 다시 수행	`calculate()` 재사용	`register_ignoresClientCalculatedValues`
2. 같은 동·삭제 안 된 구역과 이름 중복 확인	`existsByDongIdAndNameAndDeletedAtIsNull`	중복·다른 동·삭제된 이름 테스트 3개
3. 내부 점으로 center_lat/lng	`getInteriorPoint` 결과	`register_centerIsInsidePolygon`
4. zone 저장	`ZoneRepository.save`	`register_createsZone`
5. 작업 기록 (ZONE_CHANGE)	**미구현** (#6 대기)	-

명세 항목	구현	확인
409 DUPLICATE_ZONE_NAME	`DuplicateZoneNameException`	`register_duplicateNameInSameDong_conflict`
201 {id, dongId, name, areaM2}	`ZoneCreateResponse`	`register_createsZone`